DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE CODE/NAME : CCS356 OBJECT ORIENTED SOFTWARE ENGINEERING

YEAR/SEM : III / VI SEM

UNIT I - SOFTWARE PROCESS AND AGILE DEVELOPMENT			
Introduction to Software Engineering, Software Process, Perspective and Specialized Process Models –Introduction to Agility-Agile process-Extreme programming-XP Process-Case Study.			
PART-A			
1.	Define software engineering.	2	K1
2.	List the goals and characteristics of software engineering.	2	K1
3.	What is the difference between verification and validation with suitable examples?	2	K2
4.	Are the unified process and UML being the same thing? Justify your answer.	2	K2
5.	Is it possible to combine process models? If so, provide an example.	2	K2
6.	For the word processing system, which life cycle model would you choose? Give the reason why you would choose this model.	2	K3
7.	Why crystal is called a family of agile methods?	2	K2
8.	Why CI/CD is important in XP?	2	K2
9.	Differentiate between the traditional process models and agile development models with examples.	2	K2
10.	Differentiate between known risks and predictable risks.	2	K2
11.	Harry runs a grocery shop manually and now wants customized software. Draw a suitable SDLC model.	2	K3
PART B/C			
1.	Illustrate the object-oriented software engineering development activities and their products.	13	K2
2.	Explain the perspective process models, specialized process models with neat sketches.	13	K2

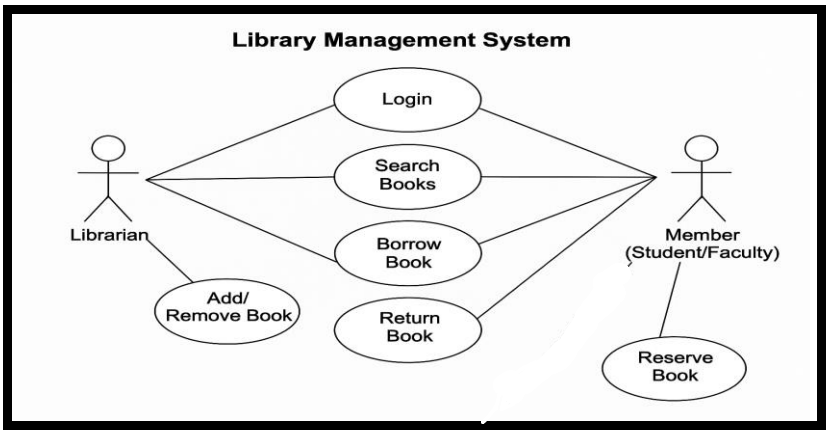
3.	Explain the principles of agile software development and its characteristics.	8+5	K2
4.	Discuss how XP supports quality delivery in projects with changing requirements and limited time.	13	K2
5.	For the situation not to define detailed input and output requirements, with minimum risk of a particular client, which process models' suits? Justify your answer.	13	K3
6.	Harry opened his Grocery store 5 years back. He has started with very little earning but after 5 years his profit has increased. till now he was managing his all-grocery shop manually but now he wants a customized software product for his shop. Develop a software development model and explain.	15	K3
7.	A software organization is developing a word processor with features equivalent to MS Word and additional market-driven requirements, to be delivered in 10 months. Analyze and compare the use of traditional and agile process models for this project.	15	K4
8.	Which is the process model that can more directly accommodate early user participation in determining, shaping, or evaluating emerging system functionality? Discuss with reasons.	13	K3
UNIT II - REQUIREMENTS ANALYSIS AND SPECIFICATION			
Requirement analysis and specification – Requirements gathering and analysis – Software Requirement Specification – Formal system specification – Finite State Machines – Petrinets – Object modelling using UML – Use case Model – Class diagrams – Interaction diagrams – Activity diagrams – State chart diagrams – Functional modelling – Data Flow Diagram- CASE TOOLS			
PART-A			
1.	What is requirement engineering and its process?	2	K1
2.	List out the various types of traceability in software engineering.	2	K1
3.	Define software prototyping.	2	K1
4.	Differentiate between functional and non-functional requirements.	2	K2
5.	Why is requirement analysis important?	2	K2

6.	State cardinality in data modeling.	2	K1
7.	Why is feedback important to the software team?	2	K2
8.	What does win-win mean in the context of negotiation during the requirements engineering activity?	2	K1
9.	How inheritance is referred to in class-based modeling?	2	K2
10.	List out some of the problems that occur when requirements must be elicited from three or four different customers.	2	K1
11.	Differentiate between the structured analysis and object-oriented strategies for requirements analysis?	2	K2
PART B/C			
1.	Describe how software requirements are collected, organized and documented? Mention the SRS template for documentation.	13	K2
2.	Explain about Petrinets and Draw the Petrinet that depicts the operation of the apartment automatic garage door.	6+7	K3
3.	Develop a requirements gathering "kit." The kit should include a set of guidelines for conducting a requirements gathering meeting and materials that can be used to facilitate the creation of lists and any other items that might help in defining requirements.	13	K3
4.	Explain the various UML diagrams used in object modelling and illustrate each with a neat diagram.	15	K2
5.	A Web-based Pothole Tracking and Repair System (PHTRS) records pothole details, work order information, and damage claims reported by citizens. Citizens can report potholes by providing location and severity details. The municipal authority reviews the report, generates work orders for repair teams, and updates the repair status. Citizens may also submit damage claims caused by potholes, which are processed by the authority. For the above scenario: a) Identify the actors and use cases and construct a Use Case Diagram for the system. b) Develop a Class Diagram showing the major classes, their attributes, methods, and relationships.	13	K3

	c) Construct an Activity Diagram to represent the workflow from reporting a pothole to completing the repair process. d) Develop a Statechart Diagram showing the different states of a pothole report from submission to closure.		
6.	Demonstrate the use of Data Flow Diagrams by drawing the context level, Level 1 and Level 2 diagrams for an online book store system.	13	K3
7.	Explain the functional modeling with a neat block diagram and illustrate how it supports requirements analysis and system design.	13	K2
8.	Apply CASE tools in a software development project and illustrate how they support different phases of the software development life cycle.	13	K3
UNIT III–SOFTWARE DESIGN			
Software design – Design process – Design concepts – Coupling – Cohesion – Functional independence – Design patterns – Model-view-controller – Publish-subscribe – Adapter – Command – Strategy – Observer – Proxy – Facade – Architectural styles – Layered - Client Server - Tiered -Pipe and filter- User interface design-Case Study.			
PART-A			
1.	What is software design? Is writing a program the same as designing software?	2	K1
2.	How do we assess the quality of a software design?	2	K2
3.	List out the data design elements and principles of software design.	2	K2
4.	Indicate the elements of the analysis model and the design model.	2	K2
5.	How can the architecture design be represented?	2	K2
6.	Outline the aspects considered when evaluating modular design quality.	2	K2
7.	Differentiate between horizontal partitioning and vertical partitioning.	2	K2
8.	What are the golden rules in user interface design?	2	K1
9.	Identify the type of coupling in the following, Part of the program handles lookup for customers. When a customer is not found, a component adds a customer by directly modifying the	2	K2

	contents of the data structure containing customer data.		
10.	How will you reduce the user's memory load in the user interface?	2	K2
PART B/C			
1.	Explain the design principles and its process with an example.	13	K2
2.	Summarize the characteristics of good software design and describe how requirements are transformed into design.	7+6	K2
3.	Explain MVC architecture and discuss the challenges and limitations in implementing MVC in software development.	8	K2
4.	Explain publish subscribe model in software design.	13	K2
5.	Discuss different architectural styles followed in system design.	13	K2
6.	Describe the concept of cohesion and coupling. State the difference between cohesion and coupling with suitable scenarios.	13	K2
7.	Describe the different design patterns used in software development and illustrate each with an example.	13	K2
8.	Describe the role of software design in large-scale software systems and illustrate it with examples.	13	K2
9.	You are developing a system that stores its data on a Unix file system. You anticipate that you will port future versions of the system to other operating systems that provide different file systems. Propose a subsystem decomposition that anticipates this change.	15	K3
UNIT IV - SOFTWARE TESTING AND MAINTENANCE			
Testing – Unit testing – Black box testing– White box testing – Integration and System testing– Regression testing – Debugging - Program analysis – Symbolic execution – Model Checking-Case Study			
PART-A			
1.	Write the steps for debugging and objectives of testing.	2	K2
2.	What are the testing principles the software engineer must apply while performing the software testing?	2	K1
3.	Do both verification and validation make use of test-case design methods and testing strategies?	2	K2

4.	Why is a highly coupled module difficult to perform unit tests?	2	K2
5.	How can project scheduling affect integration testing?	2	K2
6.	Who should perform the validation test—the software developer or the software user? Justify your answer.	2	K2
7.	Is unit testing possible or even desirable in all circumstances? Provide examples to justify your answer.	2	K2
8.	Will exhaustive testing (even if it is possible for very small programs) guarantee that the program is 100 percent correct?	2	K2
9.	Which errors tend to be more serious—client-side errors or server-side errors? Why?	2	K2
10.	Differentiate between load testing and stress testing.	2	K2
11.	What is the objective of security testing? Who performs this testing activity?	2	K1
12.	Differentiate between internal and external views of testing.	2	K2
	PART B/C		
1.	Explain the various testing methods with suitable examples.	7	K2
2.	Describe how debugging, program analysis, symbolic execution, and model checking contribute to reliable software development.	13	K2
3.	The concept of "antibugging" is an extremely effective way to provide built-in debugging assistance when an error is uncovered: a. Develop a set of guidelines for antibugging. b. Discuss advantages of using the technique. c. Discuss disadvantages.	15	K3
4.	Explain the criterion for completion of software testing.	8	K2
5.	Compare and contrast white box testing and black box testing with a banking application.	15	K2
6.	Consider the pseudo code for simple program below, Program simple subtraction Input(x,y) Output(x) Output(y) If x>y then DO x-y=z	13	K3

	Else y-x=z End If Output(z) Output “End Program” Perform basis path testing and generate test cases.		
7.	Design a set of test cases for a class MyList that includes methods such as add (), remove (), contains (), and size (). Utilize both black-box and white-box testing techniques to ensure comprehensive coverage.	13	K3
8.	Given a use case diagram for a library management system (LMS), identify possible test scenarios and develop test cases to validate the system’s behavior.  	13	K3

UNIT V - PROJECT MANAGEMENT			
Software Project Management- Software Configuration Management - Project Scheduling- DevOps: Motivation-Cloud as a platform-Operations- Deployment Pipeline: Overall Architecture Building and Testing-Deployment- Tools- Case Study			
PART-A			
1.	How do you estimate time required for a software development project?	2	K2
2.	List out the tools for quality control in project management.	2	K1
3.	Indicate the elements associated with the implementation of an effective SCM system.	2	K2
4.	Develop a checklist for use during configuration audits.	2	K2
5.	Software for system X has 24 individual functional requirements and 14 nonfunctional requirements. What is the specificity of the	2	K3

	requirements? Completeness?		
6.	Differentiate between process and project metrics.	2	K2
7.	Why should some software metrics be kept “private”? Provide examples of three metrics that should be private. Provide examples of three metrics that should be public.	2	K2
8.	Mention impact of LOC and FP in cost estimation.	2	K2
9.	List out the common cloud service models and their roles in software project management.	2	K1
10.	Differentiate between SPM and SCM.	2	K2
11.	Indicate the commonly used tools that support quality control in project management.	2	K2
12.	Recognize the tools used for testing within DevOps software practices.	2	K2
13.	Outline the concept of a Gantt chart used in project scheduling.	2	K2
14.	Indicate the role of automation in DevOps software development.	2	K2
PART B/C			
1.	Explain the software project management and configuration management in detail.	13	K2
2.	Develop a software tool that will compute cyclomatic complexity for a programming language module. You may choose the language.	13	K3
3.	A legacy system has 940 modules. The latest release required that 90 of these modules be changed. In addition, 40 new modules were added and 12 old modules were removed. Compute the software maturity index for the system.	13	K3
4.	A class X has 12 operations. cyclomatic complexity has been computed for all operations in the OO system, and the average value of module complexity is 4. For class X, the complexity for operations 1 to 12 is 5, 4, 3, 3, 6, 8, 2, 2, 5, 5, 4, 4, respectively. Compute the weighted methods per class.	13	K3
5.	Demonstrate with real-life examples of three cases in which the customer and end user are the same and three cases in which they	15	K3

	are different.		
6.	Explain in detail the COCOMO model for software cost estimation. A project is expected to have 30,000 LOC. Estimate: Effort (in person-months) Development time (in months) Cost, assuming ₹60,000 per person-month, using the Basic COCOMO model (Organic mode).	5+8	K3
7.	Given the following components of a software system: ❑ 35 Inputs ❑ 20 Outputs ❑ 10 Inquiries ❑ 6 Files ❑ 4 Interfaces Calculate the unadjusted function points assuming average complexity. Estimate the effort in person-months assuming a productivity of 12 FP/PM.	13	K3
8.	Explain the DevOps architecture in detail and discuss the role of continuous integration and continuous delivery (CI/CD).	15	K2
9.	Explain the phases of the software deployment pipeline with a neat diagram.	13	K2